ABSTRACT

In the bustling world of petrol stations, efficiency and accuracy reign supreme. The Petrol Station Management System (PSMS) emerges as a knight in shining armor, a software application wielding the power of PHP scripting and the lightweight might of an SQLite database to streamline operations and empower informed decision-making.

PSMS tackles the daily grind of petrol station management head-on. Gone are the days of tedious manual sales recording – cashiers can now seamlessly record fuel sales, capturing vital details like fuel type, quantity dispensed, and total amount. This data finds a secure home in the SQLite database, readily accessible for further analysis. Furthermore, PSMS acts as a vigilant guardian of fuel inventory. By keeping a watchful eye on current stock levels, the system empowers administrators to anticipate reorders before empty pumps bring operations to a screeching halt.

1. INTRODUCTION

Imagine a petrol station where sales recording is swift and accurate, inventory is meticulously tracked, and valuable insights are readily available. This is the reality made possible by the Petrol Station Management System (PSMS). This software application, built with PHP and a lightweight SQLite database, empowers petrol stations to operate at peak efficiency.

PSMS tackles the daily challenges faced by petrol stations head-on. It eliminates tedious manual sales recording, allowing cashiers to seamlessly capture fuel type, quantity dispensed, and total amount for each transaction. This data is securely stored in the SQLite database, forming a historical record for analysis and reporting. Moreover, PSMS acts as a vigilant inventory manager, providing real-time stock levels to administrators. This empowers them to anticipate reorders and prevent stock outs that can disrupt operations and frustrate customers.

The benefits of PSMS extend far beyond mere automation. By streamlining data entry, the system minimizes the risk of errors that often plague manual processes. This translates to increased accuracy in financial records and inventory management. Additionally, PSMS unlocks the power of data-driven insights. The system can generate reports that shed light on sales trends, fuel performance, and potential stock outs. This empowers informed decision-making, allowing station owners to maximize profitability and optimize operations.

1.1 ORGANIZATION PROFILE:

Government Arts college (Men), Krishnagiri is a renowned educational institution with a long tradition and stupendous achievements. It provides a salubrious and inspiring atmosphere to every eager aspirant who wishes to pursue any course of study to achieve upper mobility in the society with a sound career at hand. Lots of influential persons of Krishnagiri district are mostly the alumni of this college.

It disseminates quality education with time-tested values to whoever comes to its fold like a citadel of an ever-shining monarch. Ever since its inception, the college has been fashioned and fortified to produce many success personalities such as impeccable scholars, radiant tycoons and notable statesmen. In short, it is a boon to the underprivileged and downtrodden of this locality.

Situated at a strategic position on the Krishnagiri-Chennai highway, 6 km from the heart of Krishnagiri town, it serves tirelessly to the backward community like a beacon light It was founded in 1964 with a main objective to uplift the people of Krishnagiri whose chief occupation was to till the field and reap the corns not knowing much about the fruits of education and Social progress.

Hon'ble Chief Minister of Tamil Nadu His Highness M. Bakthavachalam laid the foundation stone in 1964 and it had its beginning with only four degree courses. In 1965, the degree courses;

B.A Tamil, B.A History and B.Sc., Mathematics were offered. With the munificent help of the donors such as, big shots, traders and the affluent people of this area, a new building was built for the college in 1970. In the academic year 1971-72, B.A English and B.Sc., Zoology were started. In the very next academic year B.Sc., Botany was introduced. In the academic year 1976-77, semester system was introduced. Two years after B.Sc., Physics was introduced.

The introduction of P.G Botany and P.G Mathematics in 1980-81 made the college gain up-gradation as senior grade Arts College. With the commencement of M.Phil (Botany). The college was recognized as the Research Institution by Madras University in the academic year 1984-85. In two years, the part time M.Phil (Botany) course was converted into a full time course.

In the academic year 1990-91B.Sc Chemistry was introduced. In the same academic year the college was affiliated to Periyar University as it came under the jurisdiction of the same.

In the very next year B.Sc., Chemistry was introduced. After ten years, B.Sc., Computer Science was introduced. In the academic year 2003-2004, under self-financing programme, the following courses were started; B.B.A., B.Com, B.Litt (Tamil, B.Sc., (Microbiology), B.Sc (Chemistry), M.Phil (Tamil) Full time/ Part time and Ph.D. (Tamil) Full time/ Part time. However, these so-called self-financing courses were absorbed into the regular stream in the academic year 2005-06.

In the first cycle, the college was accredited by the NAAC with the credit of C++ in Feb 2005.

Inthe academic year 2007-08, shift system was introduced in the college by the Government of Tamil Nadu. Accordingly, degree courses such as, B.Sc., Chemistry, B.Sc., Computer Science

B.Com, B.B.A and B.litt (Tamil) were started.

In the academic year 2003-04, Computer Literacy Programe (CLP) was made compulsory to all non-computer major degree students

In the academic year 2008-09, Arignar Anna Centenary block was constructed. In the same academic year Language Lab for English Department and a new Laboratory for Chemistry Department were set up. The Choice Based Credit System was introduced in the very same year as well. In the academic year 2010-11 onwards, M.Phil (Full time), Ph.D. (Full time) in Mathematics has been launched.

To the sole benefit of the aspirants of the locality, M.A English Literature, M.A. Tamil Literature, M.A. History, M.Sc., Chemistry and M.Sc., Computer Science have been started since 2012-13.

The introduction of B.A. Economics, M.Sc., Physics, M.Sc., Zoology, the Part time/ Full time and M.Phil in Zoology took place in the academic year 2013-14.

The new courses that gained entry in the academic year 2014-15 were M.Phil & Ph.D. programmes (Part time/Full time) in English, History and Physics.

In the academic year 2015-16, Ph.D. degree courses (Part Time/Full Time) were introduced in the department of Zoology.

Admirably, the very same academic year, our prestigious college was accredited with B++ grade by the NAAC, New Delhi.

M.A. (Economics) degree course was introduced in the academic year 2017-18. In the very next year, B.Sc. Chemistry (Tamil Medium), M.Phil and Ph.D. Chemistry degree courses were started.

The academic year 2019-20 witnessed the jubilant introduction of B.Sc. (Statistics) degree course. It has just stepped into its Il year of successful academic progress. The college is preparing itself to face its III cycle of NAAC accreditation that probably falls on the academic year 2021-22.

As such, the college offers 14 under graduate programmes and 9 Post Graduate programmes. Seven departments offer both part-time and full-time M.Phil and Ph.D. programmes.

The college strives to achieve the overall development of the students whoever obligingly keep pace with the ideological, philosophical and educational principles of the institution. It aims at the multi-faceted growth of the students. Besides achieving academic objectives, the institution indomitably endeavours to shape the characters of the students which are indisputably considered essential for the sustained growth of student community. Gokhale stresses the same and valued it on par with other academic achievements.

1.2 SYSTEM SPECIFICATION

1.2.1 HARDWARE SPECIFICATION

CPU Type	:	5 Computer cores 2C + 3G
Clock Speed	:	2.50 GHz
RAM Size	:	4 GB

1.2.2 SOFTWARE SPECIFICATION

Operating System	:	Windows 10 pro
Programming Language	:	PHP

2. SYSTEM STUDY

2.1 EXISTING SYSTEM

The current petrol station management system likely relies heavily on manual processes, leading to inefficiencies and potential inaccuracies. Here's a breakdown of some common scenarios:

1. **Sales Recording:** Cashiers might record sales transactions in paper notebooks or ledgers, a time-consuming and error-prone method. This data is not readily accessible for analysis or reporting.
2. **Inventory Management:** Inventory levels are likely tracked manually, often through visual inspection or basic spreadsheets. This approach lacks real-time updates and can lead to stock outs or overstocking.
3. **Reporting:** Generating reports on sales trends, fuel performance, and other business metrics might involve manual data aggregation and calculations, a cumbersome and time-consuming task. This limits the ability to make data- driven decisions.

These manual processes can create bottlenecks in operations, increase the risk of errors, and hinder analysis of valuable business data. PSMS aims to address these shortcomings by automating tasks, ensuring data accuracy, and providing tools for insightful reporting.

2.1.1 DRAWBACK

The current manual system suffers from several drawbacks that hinder the efficient operation of the petrol station. These include:

Inefficiency: Manual sales recording and inventory tracking are time-consuming, slowing down operations and potentially leading to longer customer wait times.

Inaccuracy: Errors are more likely to occur with manual data entry, leading to discrepancies in financial records and inaccurate inventory levels.

Limited Data Analysis: Manual data aggregation for reports is laborious and time- consuming, hindering the ability to gain valuable insights into sales trends, fuel performance, and other business metrics. This ultimately limits informed decision- making.

2.2 PROPOSED SYSTEM

The Petrol Station Management System (PSMS) offers a modern, automated solution to address the limitations of manual processes. Built with PHP for server-side scripting and SQLite for a lightweight database, PSMS streamlines operations and empowers data-driven decision-making.

Here's a breakdown of PSMS's key functionalities:

Automated Sales Recording: Cashiers can seamlessly record fuel sales through a user-friendly interface (optional). PSMS captures details like fuel type, quantity dispensed, and total amount, storing this data securely in the SQLite database for further analysis and reporting.

Real-Time Inventory Management: PSMS provides real-time insights into current fuel stock levels, eliminating the need for manual checks. This empowers administrators to anticipate reorders before stockouts occur, ensuring a smooth operation and preventingcustomer frustration.

Comprehensive Reporting: PSMS can generate reports on various aspects of the business. Imagine reports that reveal daily and monthly sales trends, identify top- performing fuel types, and even generate low-stock alerts. This information empowers informed decision-making, allowing station owners to optimize operations, adjust stock levels strategically, and ultimately maximize profitability.

By automating tasks, ensuring data accuracy, and providing valuable reporting tools, PSMS aims to revolutionize petrol station management, leading to increased efficiency, improved customer service, and data-driven success.

2.2.1 ADVANTAGES

Core Functionalities:

- Sales Recording: Capture fuel type, quantity dispensed, total amount, and potentially date/time of transaction.
- Inventory Management: Track current stock levels for each fuel type in real-time.
- Reporting: Generate reports on sales trends (daily, monthly, etc.), fuel performance by type, low-stock alerts, and potentially even sales forecasts.

Optional Functionalities:

- Customer Management: Manage customer information, loyalty programs (if applicable), and purchase history.
- User Authentication and Security: Secure access to the system with different user roles and permissions.
- User Interface: Provide a user-friendly interface for cashiers and administrators to interact with the system (consider both desktop and mobile options if needed).
- Payment Processing (if applicable): Integrate with payment gateways for electronic transactions.
- Pump Integration (if applicable): Connect directly to pumps for automatic data capture of dispensed fuel quantities.

3. SYSTEM DEVELOPMENT

3.1 SOFTWARE DESCRIPTION

PHP (Hypertext Preprocessor) is a widely-used open-source server-side scripting language that is primarily used for web development. Here are some key points about PHP:

Server-side Scripting:

PHP is a server-side scripting language, which means that the code is executed on the server before the HTML output is sent to the client's web browser.

Dynamic Web Content:

PHP is used to create dynamic web pages and web applications. It allows developers to generate content, collect form data, manage cookies, and more.

Popularity and Adoption:

PHP is one of the most widely-used server-side languages, with a large and active community of developers. It is used by millions of websites, including popular content management systems like WordPress, Drupal, and Joomla.

Cross-platform Compatibility:

PHP is a cross-platform language, meaning it can run on various operating systems, including Windows, macOS, and Linux.

Embedded in HTML:

PHP code can be embedded directly within HTML, allowing for the dynamic generation of web pages.

Object-Oriented Programming (OOP):

PHP supports object-oriented programming, which allows for the creation of reusable and maintainable code.

Extensive Libraries and Frameworks:

PHP has a vast ecosystem of libraries, frameworks, and tools, such as Laravel, Symfony, and CodeIgniter, which provide developers with a wide range of functionality and accelerate development.

Database Integration:

PHP seamlessly integrates with various database management systems, such as MySQL, PostgreSQL, and SQLite, enabling developers to build data-driven web applications.

Security and Performance:

PHP has built-in security features and has improved in terms of performance over the years, with the introduction of newer versions and the development of more efficient frameworks and tools.

Learning Curve:

PHP is generally considered easier to learn than some other server-side languages, making it a popular choice for beginners in web development.

Overall, PHP's versatility, ease of use, and strong community support have made it a popular choice for web development, particularly for building dynamic, database-driven websites and web applications.

WHAT IS PHP?

PHP started out as a small open source project that evolved as more and more people found out how useful it was. asmus'Lerdorf unleashed the first version of PHP way back in 1994.

- PHP is a recursive acronym for PHP: Hypertext Preprocessor"
- PHP is a a server server side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites
- It is integrated with a number of popular databases, including SQLIite, PostgreSQLite, Oracle, Sybase, Informix, and Microsoft SQLite Server.
- PHP is pleasingly zippy in its execution, especially when compiled as an Apache module on the Unix side. The SQLite server, once started, executes even very complex queries with huge result sets in record-setting time
- PHP supports a large number of major protocols such as POP3, IMAP, and LDAP. PHP4 added support for Java and distributed object architectures (COM and CORBA), making n-tier development a possibility for the first time
- PHP is forgiving: PHP language tries to be as forgiving as possible
- PHP Syntax is C-Like.

COMMON USES OF PHP

- PHP performs system functions, i.e.from files on a system it can create, open, read, write, and close them.
- PHP can handle forms, i.e. gather data from files, save data to a file, through email you can send data, return data to the user.

- You add, delete, modify elements within your database through PHP.
- Access cookies variables and set cookies.
- Using PHP, you can restrict users to access some pages of your website.

CHARACTERISTICS OF PHP

Five important characteristics make PHP's practical nature possible

- Simplicity
- Efficiency
- Security
- FIexibility
- Familiarity

In order to develop and run PHP Web pages three vital components need to be installed on your computer system.

- Web Server - PHP will work with virtually all Web Server software including Microsoft's Internet Information Server (IIS) but then most often used is freely available Apache Server. Download Apache for free here http://httpd.apache.org/download.cgi
- Database _ PHP will work with virtually all database software, including Oracle and Sybase but most commonly used is freely available SQLite database.
- PHP Parser - In order to process PHP script instructions a parser must be installed to generate HTML output that can be sent to the Web Browser. This tutorial will guide you how to install PHP parser on your computer.

PHP PARSER INSTALLATION

Before you proceed it is important to make sure that you have proper environment setup on your machine to develop your web programs using PHP.

Type the following address into your browser's address box.If this displays a page showing your PHP installation related information then it means you have PHP and Webserver installed properly. Otherwise you have to follow given procedure to install PHP on your computer.

This section will guide you to install and configure PHP over the following four platforms--

- PHP Installation on Linux or Unix with Apache
- PHP Installation on Mac OS X with Apache
- PHP Installation on Windows NT/2000/XP with IIS
- PHP Installation on Windows NT/2000/XP with Apache

Apache Configuration

If you are using Apache as a Web Server then this section will guide you to edit Apache Configuration Files.

PHP.INI File Configuration

The PHP configuration file, php.ini, is the final and most immediate way to affect PHP's functionality.

Windows IIS Configuration

To configure IIS on your Windows machine you can refer your IIS Reference Manual shipped_along with IIS.

Escaping to PHP

The PHP parsing engine neds a way to differentiate PHP code from other elements in the page. The mechanism for doing so is known as 'escaping to PHP. There
are four ways to do this -

Canonical PHP tags

The most universally effective PHP tag style is -<P>. If you use this style, you can be positive that your tags will always be correctly interpreted.

Short tags are, as one might expect, the shortest option You must do one of two things to enable PHP to recognize the tags

- Choose the --enable-short-tags configuration option when you're building PHP
- Set the short open_tag setting in your php.ini file to on. This option must be disabled to parse XML with PHP because the same syntax is used for XML tags.

ASP-style tags

ASP-style tags mimic the tags used by Active Server Pages to delineate code blocks. To use ASP-style tags, you will need to set the configuration option in your php.ini file.

HTML script tags

Commenting PHP Code

A comment is the portion of a program that exists only for the human reader and stripped out before displaying the programs result. There are two commenting formats in PHP--

- **Single-line comments-** They are generally used for short explanations or notes relevant to the local code. Here are the examples of single line comments
- **Multi-lines comments-** They are generally used to provide pseudocode algorithms and more detailed explanations when necessary. The multiline style of commenting is the same as in C.

Frame Work is collection of software or program, that trigger off easy coding and implementing the code. It helps to programmer to achieve goals in short period of time. If PHP code is integrated with frame works, you can do anything with php coding skills.

Fuel PHP

Fuel PHP works based on Model View Control and having innovative plug ins. Fuel PHP supports router based theory where you might route directly to a nearer the input uri, making the closure the controller and giving it control of further execution.

Cake PHP

Cake PHP is a great source to build up simple and great web application in an easy way. Some great feature which are inbuilt in php are input validation, SQL injection prevention that keeps you application safe and secure.

FEATURES

- Build Quickly
- No need to configure
- MIT licence
- MVC Model
- Secure

SQLite DEFINITION

A database is a separate application that stores a collection of data. Each database has e or more distinct APIs for creating, accessing, managing, searching and replicating the data it holds.

Other kinds of data stores can be used, such as files on the file system or large hash tables in memory but data fetching and writing would not be so fast and easy with those types of systems.

So nowadays, we use relational database management systems (RDBMS) to store and manage huge volume of data. This is called relational database because all the data is stored into different tables and relations are established using primary keys or other keys known as foreign keys.

A Relational DataBase Management System (RDBMS) is a software that:

- Enables you to implement a database with tables, columns and indexes.
- Guarantees the Referential Integrity between rows of various tables
- Updates the indexes automatically
- Interprets an SQLite query and combines information from various tables

RDBMS Terminology:

Before we proceed to explain SQLite database system, let's revise few definitions related to database.

Database: A database is a collection of tables, with related data

Table: A table is a matrıx with data A table in a database looks like a simple spreadsheet.

Column: One column (data element) contains data of one and the same kind, for example the column postcode

Row: A row (= tuple, entry or record) is a group of related data, for example the data of one subscription.

Redundaney: Storing data twice, redundantly to make the system faster

Primary Key: A primary key is unique. A key value cannot occur twice in one table. With a key, you can find at most one row

Foreign Key: A foreign key is the linking pin between two tables.

Compound Key: A compound key (composite key) is a key that consists of multiple columns, because one column is not sufficiently unique

Index: An index in a database resembles an index at the back of a book.

Referential Integrity: Referential Integrity makes sure that a foreign key value always points to an

existing row.

SQLite DATABASE:

SQLite is a fast, easy-to-use RDBMS being used for many small and big businesses. It is developed, marketed, and supported by SQLite AB, which is a Swedish company. SQLite is becoming so popular because of many good reasons.

- SQLite is released under an open-source license. So you have nothing to pay to use it.
- SQLite is a very powerful program in its own right. It handles a large subset of the functionality of the most expensive and powerful database packages.
- SQLite uses a standard form of the well-known SQL data language.
- SQLite works on many operating systems and with many languages including PHP, PERL, C, C++, JAVA, etc
- SQLite works very quickly and works well even with large data sets.
- SQLite is very friendly to PHP, the most appreciated language for web development.
- SQLite supports large databases, up to 50 million rows or more in a table. The default file size limit for a table is 4GB, but you can increase this (if your operating system can handle to a theoretical limit of 8 million terabytes (TB).
- SQLite is customizable. The open-source GPL license allows programmers to modify the SQLite software to fit their own specific environments
- SQLite works very well in combination of various programming languages like PERL, C, C++, JAVA and PHP. Out of these languages, PHP is the most popular one because of its web application development capabilities.
- This tutorial focuses heavily on using SQLite in a PHP environment. If you are interested in SQLite with PERL, then you can look into PERL and SQLite Tutorial

3.2FRONTEND

HTML:

HTML or Hypertext Markup Language is the standard markup language used to create web pages.

HTML is written in the form of HTML elements consisting of tags enclosed in angle brackets (like <html>). HTML tags most commonly come in pairs like <h1> and <<h1>, although some tags represent empty elements and so are unpaired, for example <img>. The first tag in a pair is the start lag, and the second tag is the end tag (they are also called opening tags and closing tags). Though not always necessary, it is best practice to append a slash to tags which are not paired with a closing tag.

The purpose of a web browser is to read HTML documents and compose them into visible or audible web pages. The browser does not display the HTML tags, but uses the tags to interpret the content of the page. HTML describes the structure of a website semantically along with cues for presentation, making it a markup language rather than a programming language.

HTML elements form the building blocks of all websites, HTML allows images and objects to be embedded and can be used to create documents by denoting structural and other items. It can embed behavior of HTML web pages. interactive forms. It provides a means to create structured semantics for text such as headings, paragraphs, lists, scripts written in languages such as links, quotes JavaScript which affect the behavior of HTML web pages.

CASCADING STYLE SHEETS (CSS):

It is a style sheet language used for describing the markup language. While most often used to style look and formatting of a document written in a web pages and interfaces written in HTML and XHTML, the language can be applied to any kind of XML, SVG and XUL CSS is a cornerstone specification of CSS style sheets to describe their presentation. XML document, including plain the web and almost all web pages use CSS styles sheets to describe their presentation.

CSS is designed primarily to enable the separation of document content from document presentation, including elements such as the layout, colors, and fonts This separation can improve content accessibility, provide more flexibility and control in the specification of presentation

characteristics, enable multiple pages to share formatting, and reduce complexity and repetition in the structural content.

CSS can also allow the same markup page to be presented in different styles for different rendering methods, such as on-screen, in print, by voice (when read out by a speech-based browser or reader) and on Braille-based, tactile devices. It can also be used to allow the web page to display differently depending on the screen size or device on which it is being viewed. While the author of a document typically links that document to a CSS file, readers can use a different style sheet, perhaps one on their own computer, to override the one the author has specified. However if the author or the reader did not link the document to a specific style sheet the default style of the browser will be applied.

JAVASCRIPT:

JavaScript is the scripting language of the Web. All modern HTML pages are using JavaScript. A scripting language is a lightweight programming language JavaScript code can be inserted into any HTML page, and it can be executed by all types of web browsers. JavaScript is easy to learn.

WHY TO USE JAVASCRIPT:

JavaScript is one of the 3 languages all web developers must learn:

1. HTML to define the content of web pages.
2. CSS to specify the layout of web pages.
3. JavaScript to specify the behavior of web pages.

Example

```
x=document.getElementById("demo"); //Find the HTML element with id="demo"
x.innerHTML = "Hello JavaScript"; //Change the content of the HTML element
```

document.getElementById() is one of the most commonly used HTML DOM methods.

OTHER USES OF JAVASCRIPT

- Delete HTML elements
- Create new HTML elements
- Copy HTML elements
- In HTML, JavaScript is a sequence of statements that can be executed by the web browser.

JAVASCRIPT STATEMENTS:

- JavaScript statements are "commands" to the browser.
- The purpose of the statements is to tell the browser what to do.
- This JavaScript statement tells the browser to write "Hello Dolly" inside an HTML element with id="demo": Semicolon;
- Semicolon separates JavaScript statements.
- Normally you add a semicolon at the end of each executable statement.
- Using semicolons also makes it possible to write many statements on one line.

JAVASCRIPT CODE:

- JavaScript code (or just JavaScript) is a sequence of JavaScript statements.
- Each statement is executed by the browser in the sequence they are written.
- This example will manipulate two HTML elements.
- Example

```
document.getElementById("demo").innerHTML="Hello
Dolly":=document.getElementById("myDIV").innerHTML="How are you?";
```

JAVASCRIPT PROPERTIES:

- Properties are the values associated with a JavaScript object.
- A JavaScript object is a collection of unordered properties.
- Properties can usually be changed, added, and deleted, but some are read only.

3.3 BACKEND

SQLITE

SQLite is a popular, open-source, self-contained, and serverless relational database management system (RDBMS). It is known for its simplicity, efficiency, and lightweight nature, making it an excellent choice for various applications, especially those with low to moderate database requirements. Here is a description of SQLite:

Key Characteristics and Features of SQLite:

Self-Contained:

SQLite is a self-contained RDBMS, which means it doesn't rely on a separate server process to function. The entire database is stored in a single file on the file system, making it highly portable and easy to manage.

Serverless:

Unlike traditional client-server databases, SQLite operates without a dedicated database server. Applications directly interact with the SQLite library, simplifying deployment and configuration.

Embedded Database:

SQLite is often used as an embedded database within applications. It can be seamlessly integrated into a wide range of programming languages, including Python, C, C++, Java, and more.

ACID Complaint:

SQLite adheres to the principles of ACID (Atomicity, Consistency, Isolation, Durability), ensuring data integrity and reliable transactions.

Transactional Support:

It supports both explicit and implicit transactions, allowing developers to group multiple database operations into a single, atomic transaction.

Small Footprint:

SQLite's small code size and minimal memory footprint make it well-suited for resource-constrained environments, such as mobile devices and IoT devices.

Cross-Platform:

SQLite is cross-platform and runs on a wide variety of operating systems, including Windows, macOS, Linux, iOS, Android, and others.

Open-Source:

SQLite is open-source software, and its source code is in the public domain. This means it is free to use and can be modified as needed, subject to the open-source licensing terms.

Widely Supported:

SQLite is supported by a vast number of programming languages and libraries. This wide adoption makes it easy to find support and resources for working with SQLite.

SQL Compatibility:

SQLite supports a subset of SQL (Structured Query Language) and offers a rich set of SQL features, including indexes, views, triggers, and more.

High Performance:

Despite its lightweight nature, SQLite is known for its high performance, making it suitable for many real-world applications, especially those with read-heavy workloads.

Data Types:

It supports a range of data types, including integers, real numbers, text, and binary data.

SQLite is commonly used in a variety of applications, such as mobile app development, desktop software, embedded systems, and local data storage for web applications. Its ease of use, efficiency, and reliability have made it a popular choice for developers who need a lightweight and embedded database system.

SQLite

4. SYSTEM ANALYSIS AND DESIGN

4.1 FEASIBLITY STUDY

Market Analysis

- Assess the current market for petrol stations in the target geographic area.
- The key competitors and their offerings.
- Analyze the target customer base and their needs/requirements.
- Forecast the potential demand for a new petrol station management system**.**

Technical Feasibility

- Evaluate the technical requirements for the system, including hardware, software, and infrastructure needs .
- Assess the availability and suitability of the required technologies.
- Identify any technical risks or constraints that could impact the system implementation.
- Provide high-level system architecture and design considerations.

Operational Feasibility

- Training requirements for system operation and maintenance.
- Examine the business processes and workflows that the system needs to support.
- Assess the organizational readiness and ability to adopt the new system.
- Identify the key stakeholders and their roles/responsibilities**.**

Financial Feasibility

- Estimate the capital and operating costs associated with developing and deploying the system.
- Analyze the potential revenue streams and cost savings enabled by the system.
- Prepare a cost-benefit analysis and projected financial statements.
- Assess the overall financial viability and return on investment (ROI).

Implementation Plan

- Outline the proposed project plan and timeline for system implementation.
- Identify the key milestones, deliverables, and resources required.
- Discuss the risk management strategies and contingency plans.
- Provide recommendations and next steps for the project.

5. MODULES AND DESCRIPTION

5.1 MODULES

Certainly, these modules represent a well-structured foundation for a student management system (SMS). Let's delve into how each module contributes to the system's functionality:

- Sales Module
- Inventory Management Module
- Reporting Module
- User Management Module
- Customer Management Module

5.2 MODULES DESCRIPTION

Sales Module

- Record fuel sales transactions. Allow selection of fuel type from a list populated with data from the database.
- Enable cashiers to enter the dispensed quantity. Optionally, access customer information for loyalty programs (if applicable).
- Calculate the total amount based on fuel type, quantity, and unit price retrevedfrom the database.
- Record the sale transaction, incluing ful ID, quantity, total amount, and custome ID (if applicable). Update the stock level for the sold fuel type in the database.

Inventory Management Module

- Track current stock levels for each fuel type in real-time.
- Display current stock levels for all fuel types, retrieved from the database.
- Allow authorized users (e.g., administrators) to update stock levels based on new deliveries or adjustments.

Reporting Module

- Generate reports on various aspects of the petrol station's business
- Provide functionalities to:
- Generate reports on daily/monthly/yearly sales trends
- Identify top-performing fuel types by sales volume or revenue.
- Analyze fuel profitability based on cost and sales prices.
- Generate reports in user-friendly formats (tables, charts, etc.)

User Management Module

- Manage user accounts and access control.
- Create and manage user accounts with different roles (cashier, administrator)
- Assign permissions to ser rtoles (le.g, cashiers can record sales,
- administrators can access all functionalities).
- Implement secure authentication for login and access control.
- Customer Management Module (Optional, for loyalty programs or credit
- accounts):

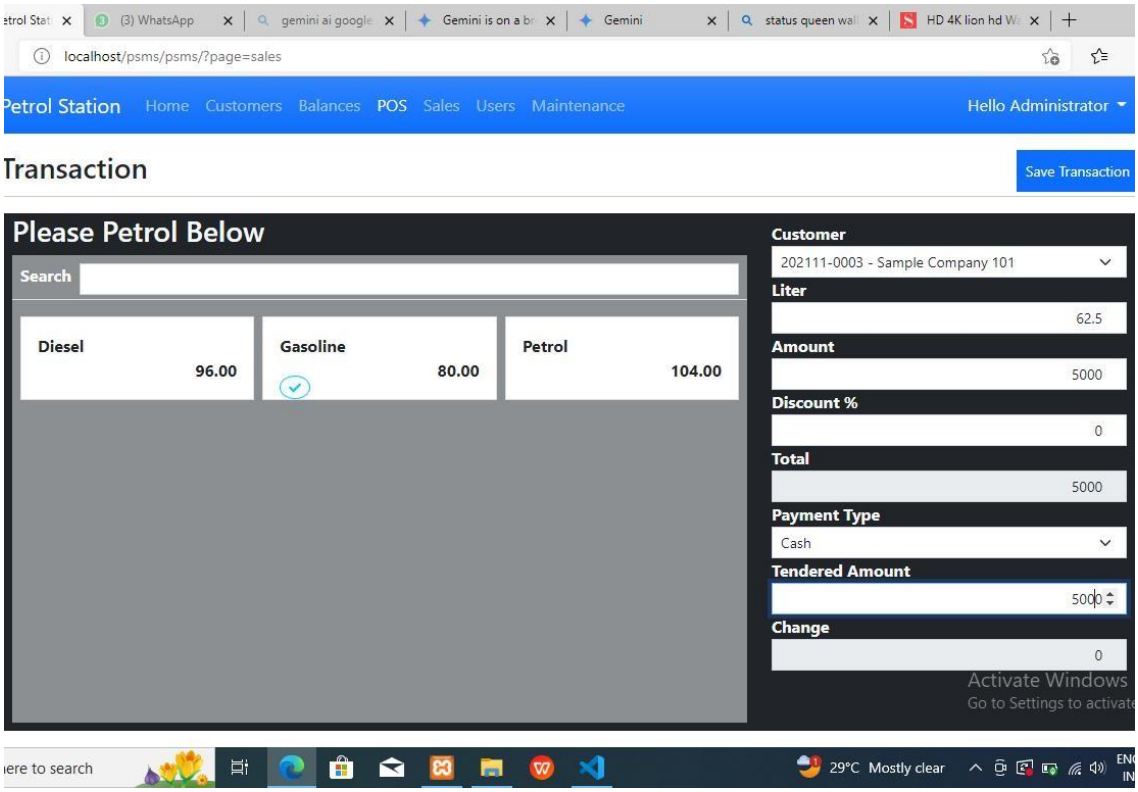
Customer Management Module:

- Manage customer information for loyalty programs or credit accounts.
- Create new customer profiles by capturing details like name, contact information, and loyalty program ID (if applicable).
- Manage customer purchase history and loyalty points.
- Track customer debts and payments (potentially integrated with the Sales Module).
- Optionally, offer functionalities for loyalty program management (e.g., points accumulation, redemption).

6.SYSTEM DESIGN

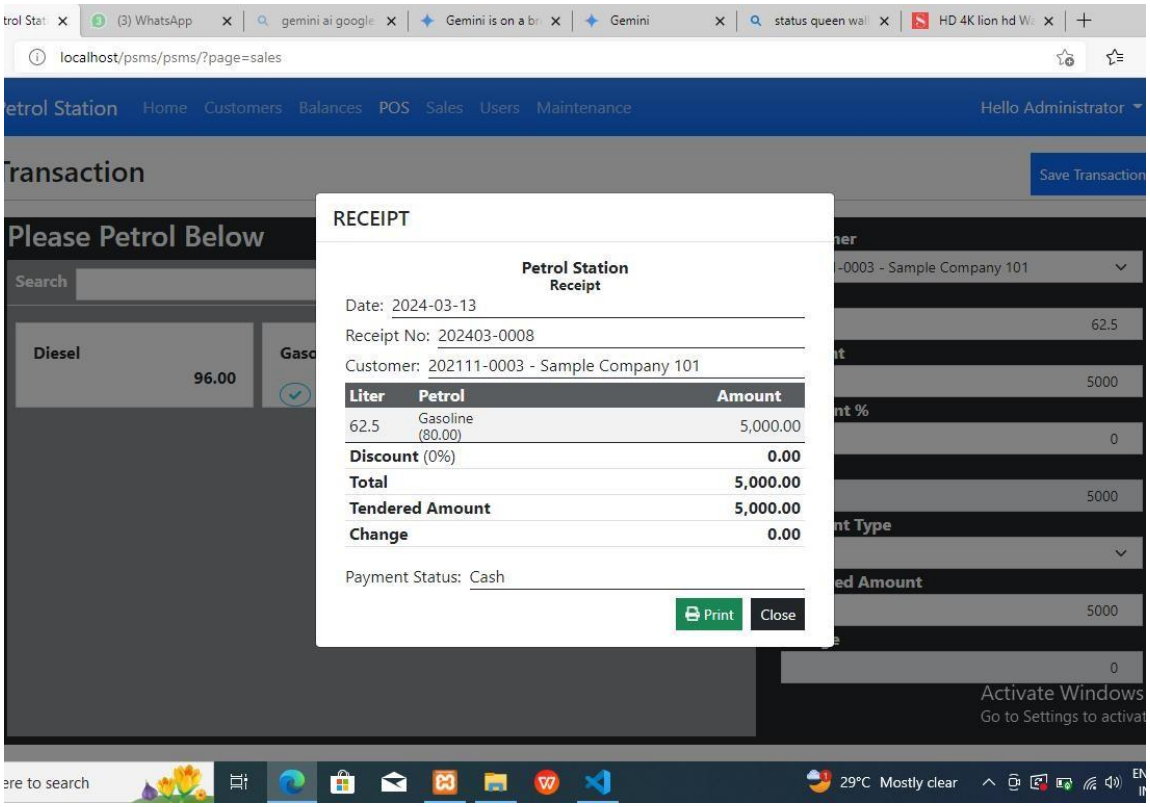
6.1 INPUT DESIGN

In PSMS, the input design focuses on capturing accurate and efficient data from cashiers. This might involve a user-friendly interface (optional) where cashiers can select the fuel type, enter the dispensed quantity, and potentially access customer information (if a loyalty program exists). Error handling can be implemented to ensure valid data entry (e.g., checking for numeric values within a reasonable range).



6.2 OUTPUT DESIGN

PSMS's output design prioritizes clear and informative displays for both cashiers and administrators. Cashiers might receive confirmation receipts for recorded sales (optional, printed or on-screen). Administrators can access reports generated by thesystem. These reports, presented in a user-friendly format (tables, charts, etc.), could visualize daily/monthly sales trends, highlight top-performing fuel types, or issue low- stock alerts, empowering data-driven decision-making.



7. TESTING AND IMPLEMENTATION

7.1 TESTING

For the petrol station management system project using PHP, you can implement the following testing strategies:

Unit Testing:

- Write unit tests for individual components and modules, such as the fuel inventory management, transaction processing, and vehicle management.
- Use a unit testing framework like PHPUnit to create and run these tests.
- Ensure that each unit test covers the expected behavior and edge cases for the specific functionality being tested.

Integration Testing:

- Test the interactions and data flow between the different components of the system, such as the frontend, backend API, and database.
- Verify that the integrated system functions as expected, with data being correctly passed between the different layers.
- Use techniques like mock objects and stubs to simulate the behavior of external dependencies during integration testing.

End-to-End (E2E) Testing:

- Perform E2E tests that simulate the complete user journey, from the frontend interface to the backend processes.
- Use a tool like Selenium or Cypress to automate the testing of the entire application flow, including user interactions, data inputs, and expected outcomes.
- Ensure that the system behaves as expected from the user's perspective, validating the overall functionality and user experience.

Performance Testing:

- Assess the system's performance under various load conditions, such as high user traffic, concurrent transactions, and large data sets.
- Use tools like Apache JMeter or Siege to simulate and measure the system's response times, throughput, and resource utilization.
- Identify and address any performance bottlenecks or scalability issues.

Security Testing:

- Evaluate the system's resistance to common web application vulnerabilities, such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
- Use tools like OWASP ZAP or Burp Suite to perform security scans and identify potential security risks.
- Ensure that the system adheres to best practices for secure coding, input validation, and authentication/authorization mechanisms.

Usability Testing:

- Involve end-users to evaluate the system's user interface, navigation, and overall user experience.
- Gather feedback on the intuitiveness, efficiency, and satisfaction of the system's workflow and functionality.
- Incorporate the user feedback to improve the system's design and usability.

Acceptance Testing:

- Collaborate with the project stakeholders to define a set of acceptance criteria that the system must meet.
- Conduct acceptance tests to validate that the system fulfills the specified requirements and meets the stakeholders' expectations.
- Document the acceptance test results and obtain sign-off from the stakeholders before deploying the system.

Regression Testing:

- Implement a comprehensive regression test suite to ensure that new changes or updates to the system do not introduce regressions or break existing functionality.
- Run the regression tests after each development iteration or before every deployment to catch any unintended issues.

7.2 IMPLEMENTATION

Implementing a petrol station management system requires careful planning, technical expertise, and a deep understanding of the industry's unique requirements. It's often advisable to work with an experienced software development partner or an industry-specific solution provider to ensure a successful deployment.

System Requirements

The key features and requirements for a petrol station management system typically include:

Fuel Inventory Management: Tracking fuel levels, deliveries, and sales to maintain optimal stock levels.

Point-of-Sale (POS) System: Handling fuel dispensing, payment processing, and customer transactions.

Fleet Management: Monitoring and managing fuel consumption for company vehicles.

Reporting and Analytics: Generating reports on fuel sales, inventory, revenue, and operational metrics.

Loyalty Program Integration: Integrating a loyalty program to attract and retain customers.

Pricing and Promotions: Configuring fuel prices and managing promotional offers.

Employee Management: Tracking employee shifts, sales performance, and attendance.

Integrations: Connecting with external systems like accounting, inventory management, or enterprise resource planning (ERP) software.

Implementation Steps

Plan and Design: Define the system's scope, identify key stakeholders, and map out the system's architecture and data flows.

Hardware Setup: Install fuel dispensers, POS terminals, servers, and networking equipment at the petrol station.

Software Development: Build or customize the management system software to meet the specific requirements, including the user interface, database, and business logic.

Data Migration: Migrate existing fuel inventory, customer, and transaction data from any legacy systems into the new management system.

Integration: Connect the management system with external systems, such as accounting software, loyalty programs, and enterprise resource planning (ERP) systems.

User Training: Provide comprehensive training to staff on the system's features and functionalities, including fuel dispensing, inventory management, and reporting.

Pilot Testing: Conduct a pilot deployment at one or more petrol stations to validate the system's functionality and make any necessary adjustments.

Full Deployment: Roll out the management system to all petrol stations, ensuring a smooth transition and continuous monitoring for any issues.

Maintenance and Upgrades: Regularly maintain the system, apply software updates, and plan for future enhancements to keep up with evolving business needs.

Key Considerations

Scalability: Ensure the system can handle growing transaction volumes and multiple petrol station locations.

Security: Implement robust security measures to protect customer data, fuel inventory, and financial transactions.

Compliance: Ensure the system complies with industry regulations, such as fuel quality standards and environmental guidelines.

User Experience: Prioritize a user-friendly interface for both staff and customers to streamline operations and enhance the customer experience.

Reporting and Analytics: Develop comprehensive reporting capabilities to support data-driven decision-making and operational optimization.

8. CONCLUSION

This document has outlined the design of a Petrol Station Management System (PSMS). The system provides functionalities for recording sales transactions, managing fuel inventory, generating reports, and potentially managing customer information (depending on specific needs). The architecture diagram provides a clear separation of concerns, promoting maintainability and scalability. The code snippets showcased essential functionalities and database interactions.

In conclusion, a well-designed petrol station management system streamlines daily operations and boosts profitability. Inventory control becomes effortless, with real-time data on fuel levels and sales. This allows for optimized ordering, preventing stockouts and overfilling of tanks. Additionally, the system strengthens financial management through accurate sales tracking, automated reports, and improved cash flow visibility. By facilitating informed decision-making and minimizing human error, a petrol station management system empowers businesses to thrive in a competitive market.

Future scope

The PSMS can be further enhanced with several features,

Advanced Reporting:

- Sales trends over time (daily, weekly, monthly, yearly)
- Top-selling fuel types
- Revenue and profit analysis
- Customer purchase history and loyalty program details

Mobile App Integration:

- Develop a mobile application for cashiers to access the PSMS on the go, record sales, and view basic reports.
- Implement features for customer self-service, allowing them to pay for fuel purchases directly from their phones.

Pump Integration:

- Explore integration with fuel pumps to automatically capture dispensed fuel quantities and update the inventory system in real-time.
- Consider security measures to prevent unauthorized access and manipulation of pump data.

Customer Loyalty Program:

- Design and implement a loyalty program to reward returning customers.
- Offer features like points accumulation, redemption for discounts or free fuel, and personalized promotions.

BIBLIOGRAPY

[1] Journal article: Okemiri, M. N., & Olumuyiwa, O. O. (2016). Examining the adoption of petrol station management system: a systematic literature review. International Journal of Scientific & Technology Research, 5(12), 182-187.

[2] Conference paper: Ahmed, A., Hassan, M. I., & Shakir, M. Z. (2014, December). A New Automation Approach for Fuel Station Management System. In 2014 International Conference on Computing, Communication and Security (ICCCS) (pp. 1-5). IEEE.

[3] Gilfillan, I. (2003). The Database Relational Model: A Practical Guide for IT Professionals. Wiley.

[4] Lerdorf, R., Tatroe, K., & MacIntyre, P. (2013). Programming PHP. O'Reilly Media.

[5] Porebski, L., Luczak, A., & Ostrowski, B. (2016). Mastering Laravel. Packt Publishing.

[6] Sharan, R. (2017). PHP 7 Real-World Programming. Packt Publishing.

[7] Welling, L., & Thomson, L. (2017). PHP and MySQL Web Development (5th ed.). Addison-Wesley Professional.

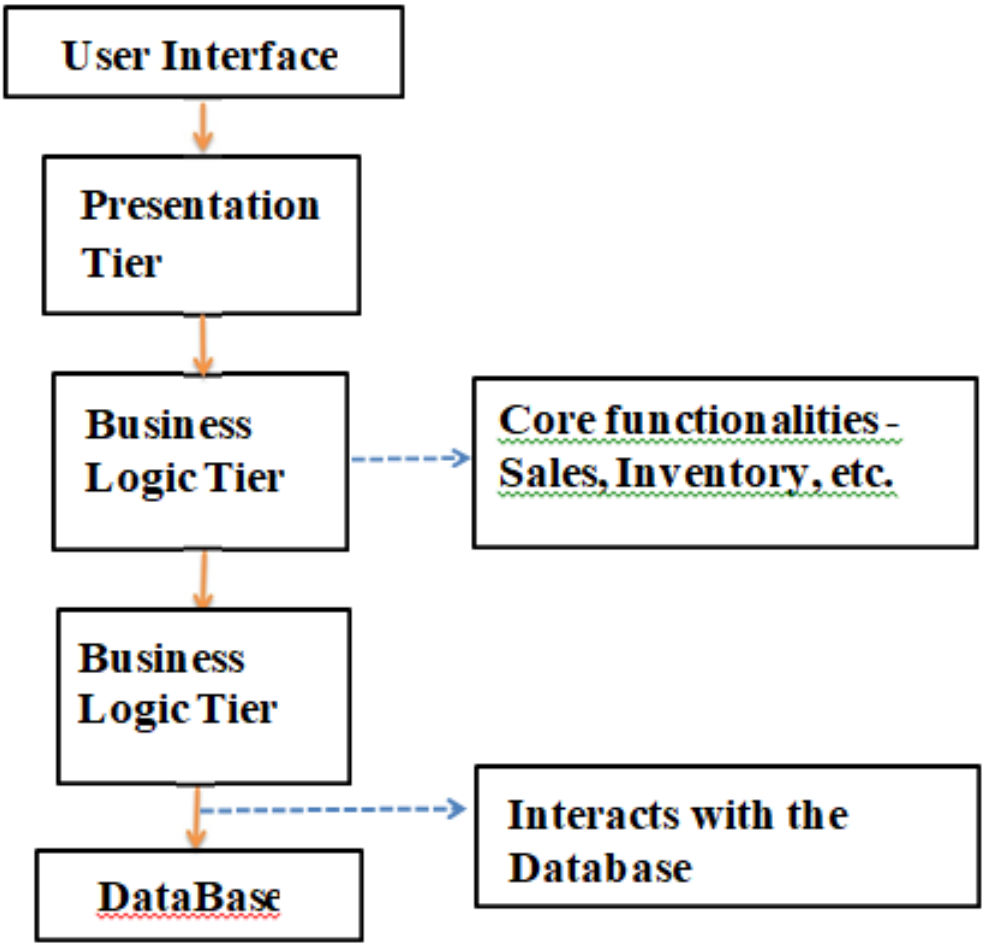
[8] Hussain, S., Ahmad, F., & Amin, R. (2012). Automated Petrol Pump Management System. In Proceedings of the 2012 International Conference on Computer, Information and Telecommunication Systems (CITS) (pp. 1-5). IEEE.

[9] Qureshi, S., Memon, N., Memon, M., & Memon, A. (2014). Automated Petrol Pump Management System. In Proceedings of the 2014 International Conference on Robotics and Emerging Allied Technologies in Engineering (iCREATE) (pp. 153-158). IEEE.

[10] Salunkhe, A., Netke, A., Patil, A., & Sharma, P. (2017). Petrol Pump Management System using Android Application. In Proceedings of the 2017 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS) (pp. 1-5). IEEE.

APPENDIX

A. FLOW DIAGRAM



B. SAMPLE CODE

Database Connection

```
<?php



class Database {



  private $db;



  public function__construct($db_name) {

    $this->db = new PDO('sqlite:' . $db_name);

    $this->db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

  }



  // ... other methods for database operations (prepared statements recommended)

}
```

Sales Module

```
<?php
```

```
class Sales {

  private $db;

  public function__construct(Database $db) {

    $this->db = $db;

  }



  public function recordSale($fuel_id, $quantity, $total_amount, $customer_id = null) {

    $stmt = $this->db->prepare("INSERT INTO sales (fuel_id, quantity, total_amount, customer_id) VALUES (?, ?, ?, ?)");
    $stmt->bindParam(1, $fuel_id, PDO::PARAM_INT);

    $stmt->bindParam(2, $quantity, PDO::PARAM_FLOAT);

    $stmt->bindParam(3, $total_amount, PDO::PARAM_FLOAT);

    $stmt->bindParam(4, $customer_id, PDO::PARAM_INT);

    $stmt->execute();



    return true;

  }
```

```
  // ... other methods for generating receipts, etc.

}
```

Inventory Management Module

```
<?php


class Inventory {


  private $db;

  private $unit_price;


  public function__construct(Database $db, $fuel_id, $initial_stock) {

    $this->db = $db;

    $this->getUnitPrice($fuel_id); // Fetch unit price from database

    $this->updateStockLevel($fuel_id, $initial_stock); // Set initial stock level

  }
```

```
public function updateStockLevel($fuel_id, $adjustment) {

  // ... prepare and execute statement to update stock level in database

}



public function getCurrentStock($fuel_id) {

  // ... prepare and execute statement to retrieve current stock level from database

}

public function getUnitPrice($fuel_id) {
  // ... prepare and execute statement to retrieve unit price from database

  $stmt = $this->db->prepare("SELECT price_per_unit FROM fuels WHERE id = ?");

  $stmt->bindParam(1, $fuel_id, PDO::PARAM_INT);

  $stmt->execute();

  $result = $stmt->fetch(PDO::FETCH_ASSOC);

  $this->unit_price = $result['price_per_unit'];

}
```

```
  // ... other methods for managing low-stock alerts

}
```

User Management Module

```
<?php



class User {



  private $id;

  private $username;

  private $password; // Hashed password storage recommended
  private $role; // e.g., "cashier", "admin"


  // ... methods for user creation, login, access control based on roles

}
```

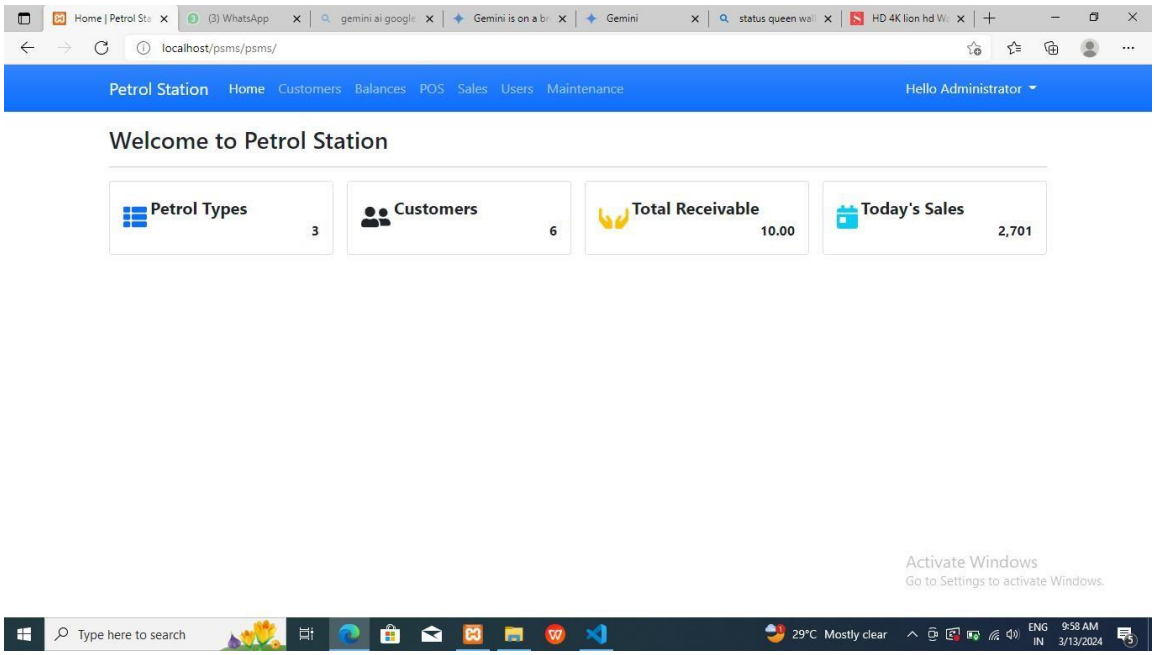
Report Module

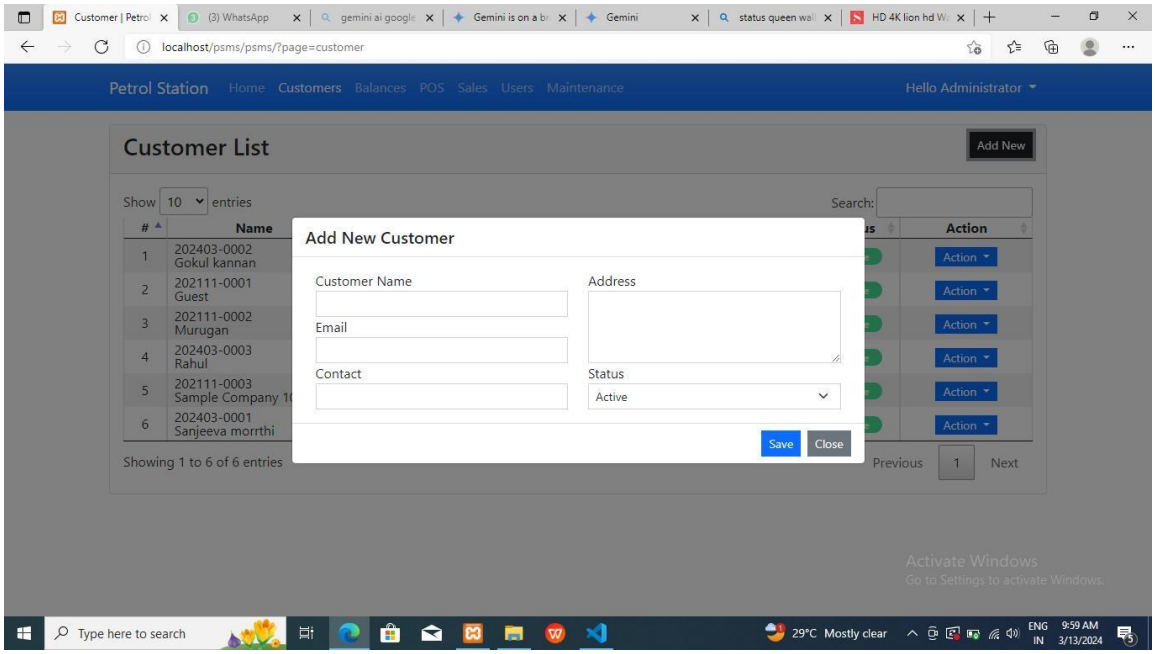
```
<?php



class Report {



  private $db;



  public function__construct(Database $db) {

    $this->db = $db;

  }



  public function getDailySalesReport($date) {

    $stmt = $this->db->prepare("
      SELECT
        f.name AS fuel_name,
      SUM(s.quantity) AS quantity_sold,
      SUM(s.total_amount) AS total_revenue
      FROM sales s
      INNER JOIN fuels f ON s.fuel_id = f.id
      WHERE DATE(s.created_at) = ?
      GROUP BY f.id
```

```
    ");
    $stmt->bindParam(1, $date, PDO::PARAM_STR);

    $stmt->execute();

    return $stmt->fetchAll(PDO::FETCH_ASSOC);

  }

  // ... additional methods for generating reports on different date ranges,

  // including functionalities for sales by fuel type, revenue breakdown, etc.



}
```

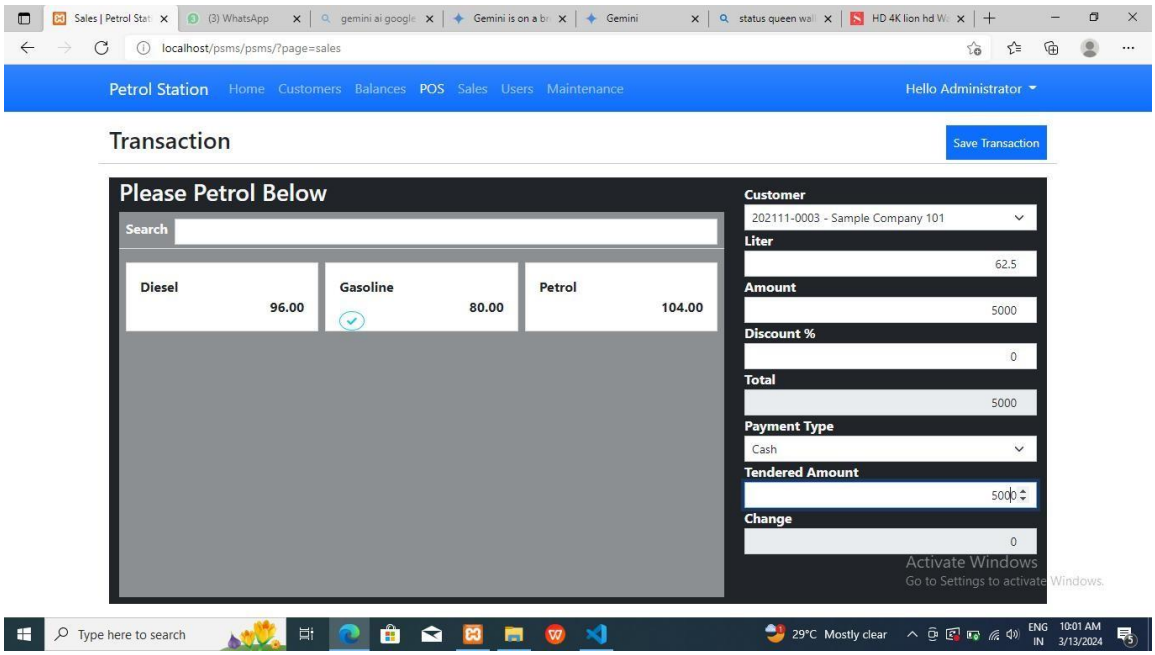
C. SAMPLE INPUT

Home Page





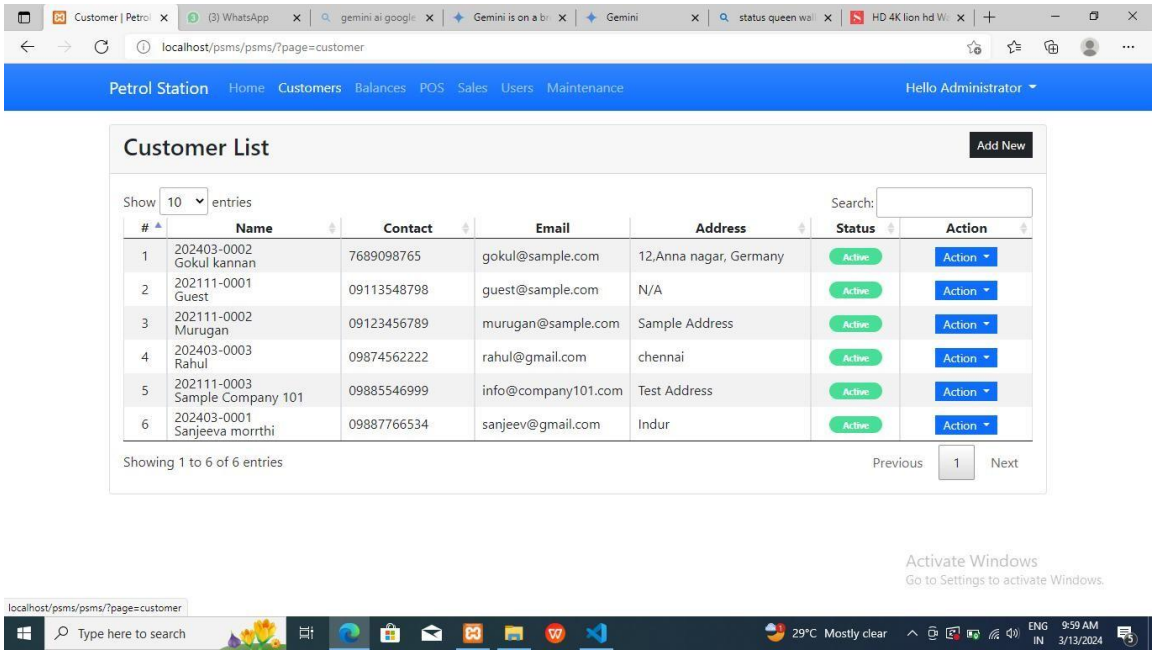
Add Customer



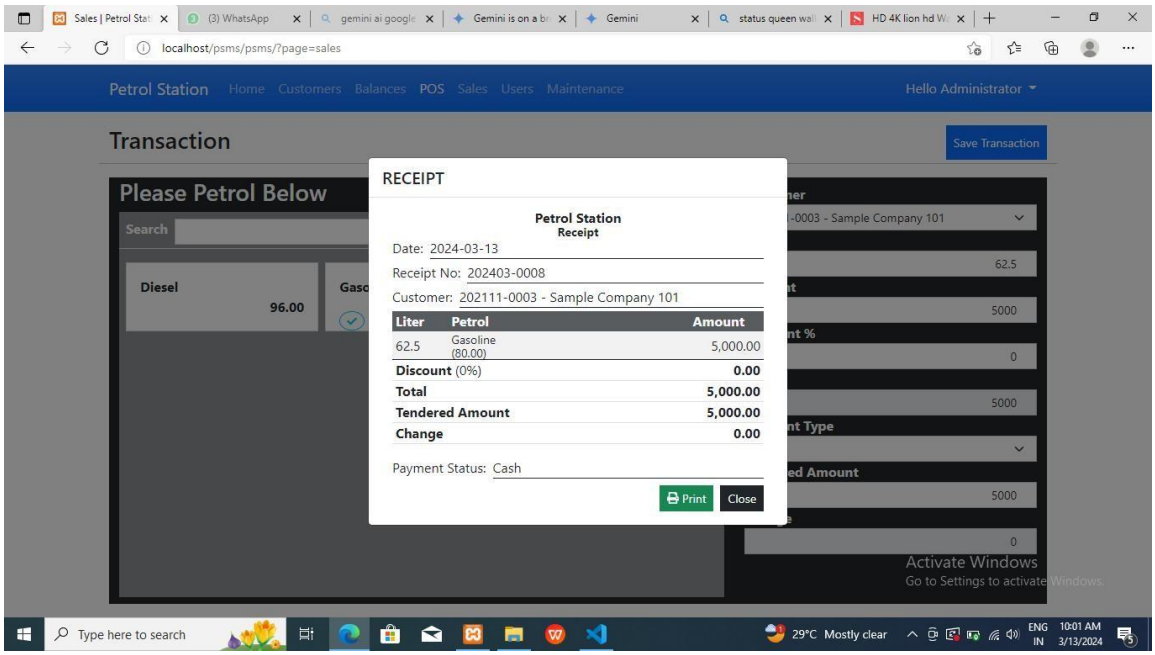
POS

D. SAMPLE OUTPUT

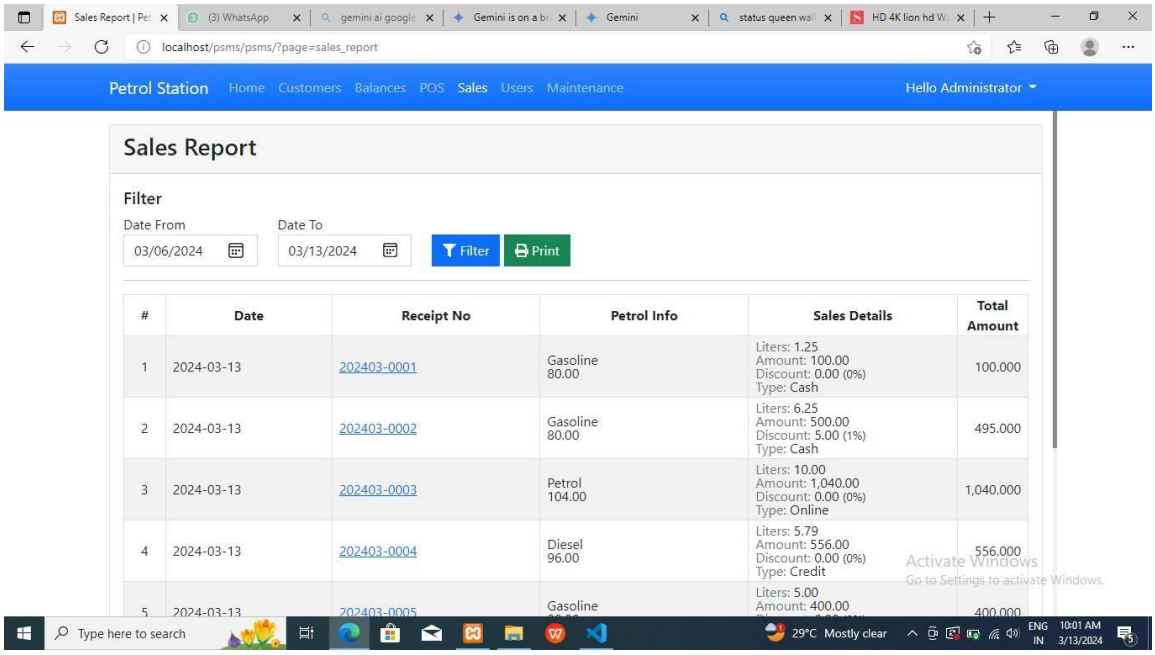
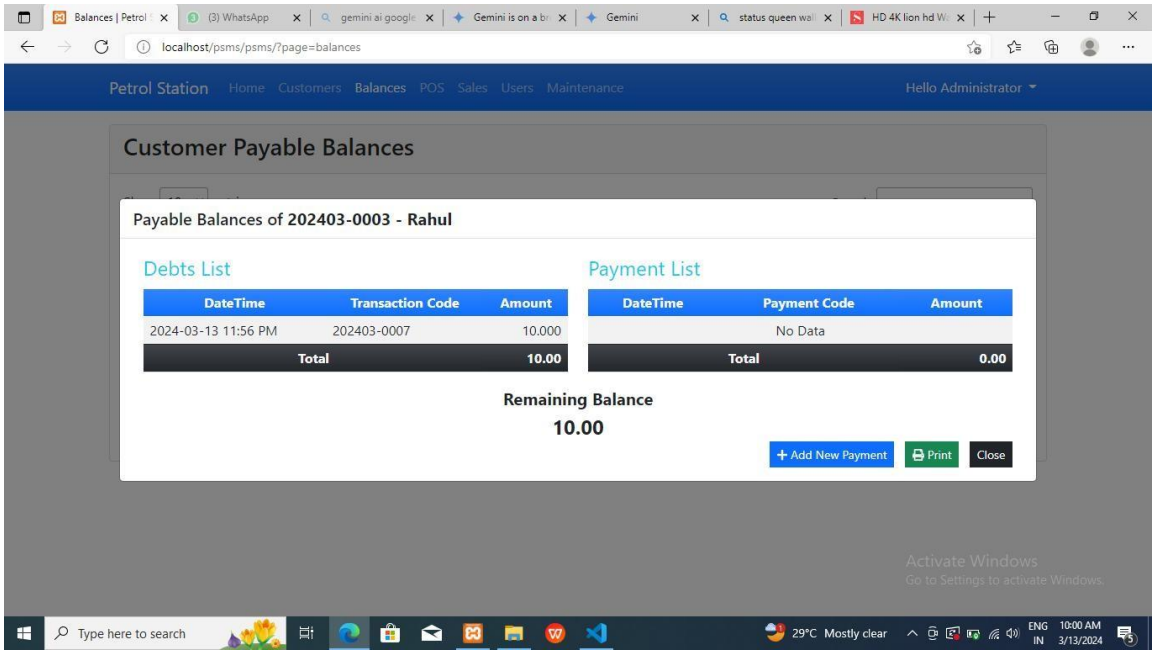
Customer List



POS Receipt



Sales Report



Customer Payable Balances